

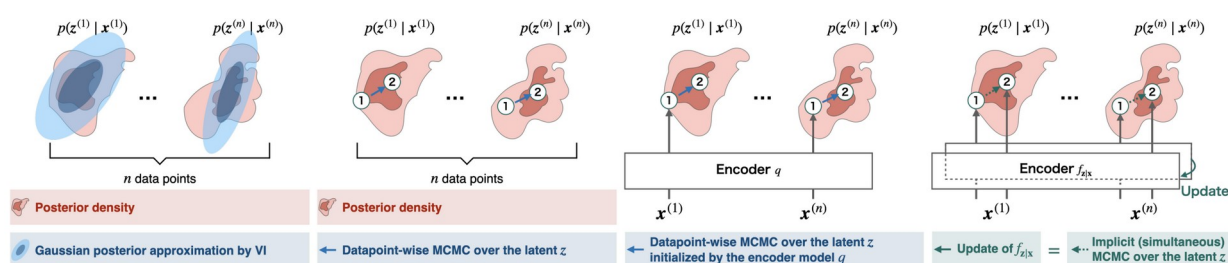
NeurIPS 2022: Moving the Randomness from the Latents to the Encoder

The University of Tokyo's Langevin Autoencoder amortizes MCMC the way a VAE amortizes inference — and beats the VAE on test likelihood

Train a deep latent variable model (DLVM) by maximum likelihood and you immediately hit the same wall everyone hits: the objective contains an expectation over the posterior $p(z|x)$, and that posterior almost never has a closed form. How you approximate it has, for years, split deep generative modeling into two camps.

The first camp is variational inference (VI) — the machinery behind the VAE. An encoder network predicts the latents for every data point in a single forward pass, and that amortization is what makes training fast. The price is expressiveness: the encoder can only emit a tractable distribution, usually a Gaussian, so the quality of the approximation is capped by that family no matter how much you train. The second camp is Markov chain Monte Carlo (MCMC), and Langevin dynamics in particular. MCMC samples from the true posterior without assuming its shape, so it can be arbitrarily flexible. Its problem has always been speed: the textbook version runs a separate sampling chain for every single data point, and it converges slowly, which is why it has rarely been practical for training deep models.

This NeurIPS 2022 paper from the University of Tokyo asks whether you can have both — the amortized efficiency of VI and the sampling flexibility of MCMC. Its answer is the Langevin Autoencoder (LAE), built on a new sampling algorithm the authors call Amortized Langevin Dynamics (ALD). The one-line intuition: take the noise that Langevin sampling injects into the latent variables, and move it onto the encoder's parameters instead.



(a) Variational Inference (b) Langevin Dynamics (c) Hoffman [2017] (d) ALD (ours)

Figure 1. Four ways to approximate the posterior. (a) Variational inference fits a tractable family such as a Gaussian; (b) traditional Langevin dynamics runs gradient-based sampling iterations directly on the latents, per data point; (c) Hoffman [2017] uses an encoder to initialize those Langevin chains but still iterates per data point; (d) ALD also uses an encoder, but treats its output directly as a posterior sample and performs the update implicitly by updating the encoder's parameters.

Links:

- Paper: <https://arxiv.org/abs/2209.07036>
- Code: <https://github.com/iShohei220/LAE>

The problem: MCMC is more accurate, so why doesn't anyone train with it?

Start with the tension. Gradient-based maximum-likelihood learning needs an expectation over $p(z|x)$, and that posterior has no closed form. VI answers this by fitting a tractable stand-in distribution: one encoder pass yields latents for the whole dataset, but the stand-in's shape is fixed in advance — a Gaussian stays a Gaussian, and it struggles the moment the true posterior is multimodal or strongly correlated.

MCMC takes the opposite route. It assumes nothing about the posterior's shape and samples from it directly, so in principle it can approximate an arbitrarily complex posterior. Langevin dynamics is one such method: it follows the gradient of the log-posterior and adds a dose of Gaussian noise, nudging samples toward high-probability regions step by step. The catch is its per-datapoint nature — every new observation needs its own full sampling chain, and with thousands of training points that cost is prohibitive. Hybrids like Hoffman [2017] use an encoder to give each chain a better starting point, but the sampling iterations themselves are still run per data point, a cost paid at both training and test time.

The method: let the Langevin noise act on the encoder's parameters

The key move in ALD is to change what the randomness acts on. Traditional Langevin runs a stochastic differential equation (SDE) on each latent z ; ALD instead defines a deterministic encoder $f(x; \phi)$ that maps observations to latents, and runs the Langevin equation on the encoder's parameters ϕ . Because the latents are produced by the encoder, dynamics on ϕ induce dynamics on the latent space — so the encoder's outputs can be read off directly as posterior samples.

Written out, the update on the parameters is $d\phi = -\nabla_{\phi} V(\phi) dt + \sqrt{2} dB$, where the potential $V(\phi) = \sum U(x^{(i)}, f(x^{(i)}; \phi); \theta)$ sums the negative log-joint over all data points. Discretized, each step is a draw from a Gaussian: $\phi' \sim N(\phi - \eta \nabla_{\phi} V(\phi), 2\eta I)$. The important detail is that every data point shares the same parameters ϕ , so a single update advances the sampling for all of them at once — which is exactly what amortization means here.

Moving the noise onto the parameters isn't free, though: you need a guarantee that the latents you read off are actually distributed according to the posterior you wanted. The paper's Theorem 1 supplies the condition. When the encoder has the form $f(x; \Phi) = \Phi g(x)$ — a fixed feature extractor g followed by a trainable last linear layer Φ — and the width d of that layer is at least the number of data points n in the batch, the latent samples induced by ALD converge to the true posterior. Two clean special cases fall out: traditional Langevin dynamics is a special case of ALD, and the ordinary autoencoder is a special case of the LAE.

The Langevin Autoencoder simply folds this sampler into the training loop. Before each decoder update, it runs T ALD steps on the encoder's last layer to draw posterior samples, optionally followed by a Metropolis-Hastings (MH) rejection step that removes the bias introduced by discretization. In code it looks almost identical to a standard autoencoder — the only difference is that the encoder takes a few noisy update steps along the way.

That $d \geq n$ condition is not just theory. In a bivariate-Gaussian toy problem, the authors vary the input dimensionality d of the encoder's last layer and watch what happens to the samples.

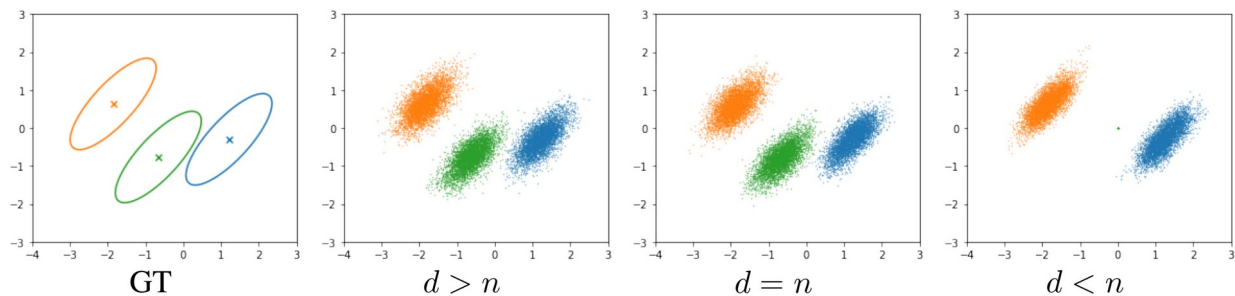


Figure 2. Effect of encoder capacity in the bivariate-Gaussian toy example; d is the input dimensionality of the encoder's last linear layer. When $d > n$ and $d = n$, the samples for all three data points cover their true posteriors (GT); once $d < n$, the middle data point's samples collapse to a single point — a direct confirmation of the rank condition $d \geq n$ in Theorem 1.

Evidence I: ALD reaches posteriors that VI cannot

Where does the flexibility actually show up? The paper uses a "neural likelihood" toy example: the likelihood is defined by a randomly initialized neural network, engineering a posterior that is both multimodal and strongly correlated, and then asks which methods can recover it. Shapes like this are precisely where a Gaussian stand-in breaks down.

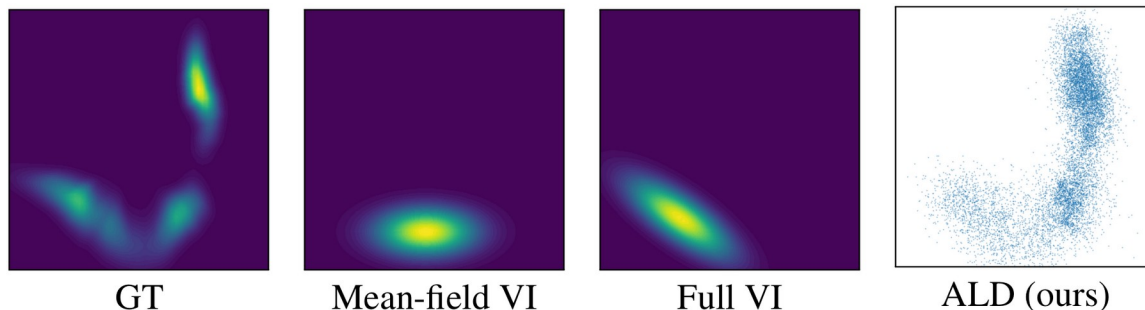


Figure 3. Posterior comparison on the neural-likelihood example. From left to right: the ground-truth posterior (GT), mean-field VI, full VI, and samples from ALD. Mean-field VI captures only one mode, full VI manages a single correlated ellipse, while ALD's samples reproduce the multimodal, curved structure of the true posterior.

The read is straightforward. Mean-field VI collapses the whole posterior into one isolated mode; full VI does a little better but still amounts to a single tilted ellipse — both discard the true multimodal, curved structure. ALD's samples, by contrast, trace the actual shape. That is the flexibility of genuine MCMC coming back into a setting where the variational family simply could not reach.

Evidence II: a more accurate posterior buys a better generative model

The toy example shows ALD samples accurately, but does that accuracy pay off when training a real deep generative model? The paper runs image generation on four standard datasets — MNIST, SVHN, CIFAR-10, and CelebA. For a fair comparison every method uses the same fully-connected networks, and quality is measured by the negative ELBO (evidence lower bound) per dimension, averaged over three seeds; lower is better.

Table 1. Image-generation results on four datasets, measured by negative ELBO per dimension (mean over three seeds, lower is better). LAE achieves the lowest value on all four datasets, consistently ahead of the VAE, VAE-flow, and Hoffman [2017].

Method	MNIST	SVHN	CIFAR-10	CelebA
VAE	1.189	4.442	4.820	4.671
VAE-flow	1.183	4.454	4.828	4.667
Hoffman [2017]	1.189	4.440	4.831	4.662
LAE (ours)	1.177	4.412	4.773	4.636

LAE posts the lowest negative ELBO on all four: MNIST drops from the VAE's 1.189 to 1.177, CIFAR-10 from 4.820 to 4.773, CelebA from 4.671 to 4.636, and SVHN from 4.442 to 4.412. The absolute gains are modest, but the direction is consistent — more accurate posterior sampling does translate into a better-trained generative model. The cost is wall-clock: LAE trains about 2.24× slower than a plain VAE, though it runs at essentially the same speed as the sampling-based Hoffman [2017] baseline. And all of this uses just $T = 2$ ALD iterations.

What's doing the work: the MH step and the iteration count T

If $T = 2$ is enough, how much does T matter at all — and is the MH rejection step worth its cost? The paper answers with learning curves of the negative ELBO on MNIST's test set.

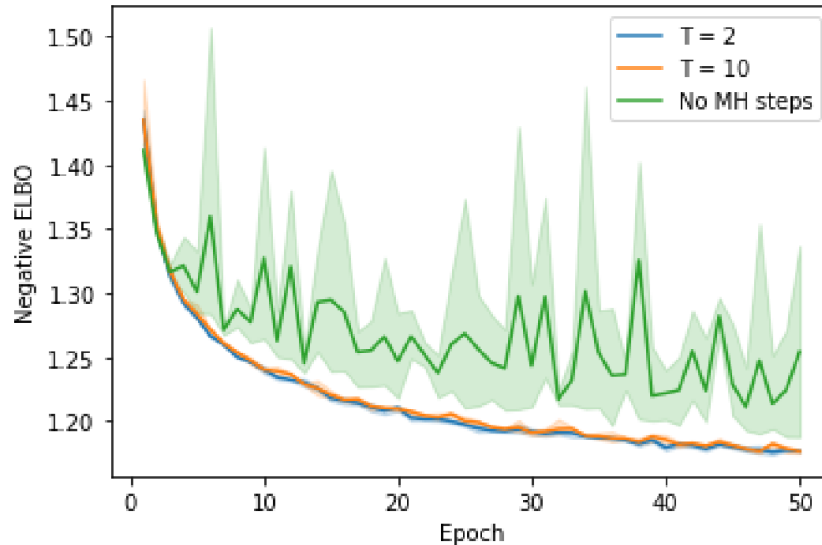


Figure 4. Learning curves of the negative ELBO on MNIST's test set. The $T = 2$ and $T = 10$ curves nearly coincide, so beyond two steps more ALD iterations barely help; dropping the MH rejection step (No MH steps) makes the curve oscillate wildly, showing that the MH step is important for stabilizing training.

Two things stand out. First, the $T = 2$ and $T = 10$ curves sit almost on top of each other, so once ALD runs at least two steps, adding more brings little — which is why the image experiments use only $T = 2$. Second, remove the MH rejection step and the curve starts swinging violently; training visibly destabilizes. The MH step costs a bit of extra computation but is what keeps training stable. Together with Figure 2, these two ablations pin down the method's behavior from both ends — the theoretical condition ($d \geq n$) and the engineering details (MH, T).

Takeaway: the autoencoder, reread as a bias-corrected sampler

The most elegant thing about this work is the rearrangement of where the randomness lives. Move the Langevin noise from the latents to the encoder's parameters, and sampling becomes both efficient — because it is amortized across all the data — and flexible — because it is genuine MCMC rather than a constrained Gaussian. The resulting LAE is a provably valid MCMC method, looks almost exactly like a standard autoencoder in code, and still beats the VAE on test likelihood.

Seen from another angle, it reframes the familiar autoencoder as a sampling-based, bias-corrected alternative to variational inference. The boundaries are worth stating plainly: the convergence guarantee relies on a specific encoder structure — a fixed feature extractor with a trainable linear last layer and $d \geq n$ — the experiments center on fully-connected networks and these standard datasets, the absolute gains are gentle, and you pay roughly 2.24× the training time. It reads less as a finished replacement and more as an opened door: MCMC, for the first time, genuinely amortized into the training of deep generative models.